System przetwarzania danych oparty na pojedynczym oraz współpracujących modelach LLM

Oliwier Komorowski Jakub Nowak

31 października 2026

Streszczenie

Projekt inżynierski dotyczy współpracy wielu dużych modeli językowych (LLM) w rozwiązywaniu złożonych problemów. Modele LLM są zazwyczaj wykorzystywane pojedynczo, jako narzędzia do redagowania tekstów, pozyskiwania informacji i tworzenia treści. W projekcie zbadano natomiast, czy połączenie kilku modeli w jeden system pozwala uzyskać lepsze rezultaty niż praca pojedynczego modelu. Aplikacja MLLMS, powstała w ramach projektu, jest inspirowana występującym w przyrodzie umysłem rojowym (ang. hive mind). Została napisana w języku Python i łączy się za pomocą API Ollamy z modelami uruchamianymi lokalnie. Obsługa odbywa się z poziomu terminala w postaci interfejsu tekstowego, do którego komunikacji nie użyto żadnej dodatkowej biblioteki. Modele współdzielą jedną wspólną przestrzeń, w której wymieniają się spostrzeżeniami i wynikami obliczeń. Wygenerowane przez nie odpowiedzi są łączone w jeden ciąg tekstowy, który stanowi kontekst dla kolejnych kroków rozumowania. Dzięki temu każdy model ma dostęp do wniosków pozostałych i może na nich budować własne odpowiedzi. Eksperymenty wykazały... Na tej podstawie sformułowano najlepsze praktyki dotyczące projektowania i implementacji systemów opartych na współpracujących LLM. Projekt ukazuje potencjał zespołowej pracy modeli językowych, stanowiąc podstawę do dalszych prac nad udoskonalaniem podobnych rozwiązań.

Słowa kluczowe: duży model językowy (LLM), współpraca wielu modeli, umysł rojowy, Ollama
Dziedzina nauki i techniki, zgodnie z wymogami OECD: Nauki o komputerach i informatyka

Abstract

This engineering project concerns the cooperation of multiple large language models (LLMs) in solving complex problems. LLMs are usually used individually, as tools for editing texts, retrieving information and creating content. This project, however, investigated whether combining several models into a single system can yield better results than a single model working alone. The MLLMS application, developed as part of the project, is inspired by the hive mind found in nature. It was written in Python and connects through the Ollama API to locally run models. The application is operated from the terminal through a text user interface, for which no additional library was used. The models share a common space in which they exchange insights and calculation results. The responses they generate are merged into a single string, which serves as context for subsequent reasoning steps. As a result, each model has access to the conclusions of the others and can build its own answers on them. The experiments showed... On this basis, best practices for designing and implementing systems based on cooperating LLMs were formulated. The project demonstrates the potential of teamwork among language models and provides a foundation for further work on improving similar solutions.

Keywords: large language model (LLM), multi-model collaboration, hive mind, Ollama
Field of science and technology in accordance with OECD requirements: Computer Science and Information technology

Spis treści

1 Wstęp i cel pracy **4**
- 1.1 Motywacja . . . 4
  - 1.1.1 Dynamiczny rozwój dziedziny . . . 4
  - 1.1.2 Nowe możliwości . . . 5
- 1.2 Cel pracy . . . 6
- 1.3 Struktura pracy . . . 7

2 Wprowadzenie do dziedziny **8**
- 2.1 Generatywna sztuczna inteligencja . . . 8
  - 2.1.1 Definicja i miejsce generatywnej sztucznej inteligencji . . . 8
  - 2.1.2 Zasada działania i zastosowania . . . 8
  - 2.1.3 Ograniczenia generatywnej sztucznej inteligencji . . . 9
- 2.2 Duże modele językowe . . . 9

3 Technologie i narzędzia **10**
- 3.1 Wybrane języki programowania . . . 10
- 3.2 Biblioteki . . . 10
- 3.3 Wybrane modele językowe . . . 10
- 3.4 Narzędzia . . . 11
- 3.5 Wymagania sprzętowe . . . 12

4 Projekt systemu **13**
- 4.1 Implementacja systemu przetwarzania z wykorzystaniem pojedynczego modelu LLM . . . 13
- 4.2 Rozszerzenie systemu do wariantu z dwoma współpracującymi modelami LLM 13

5 Badania i analiza **14**
- 5.1 Wybór benchmarków oraz metryk oceny jakości odpowiedzi . . . 14
  - 5.1.1 Benchmarki . . . 14
  - 5.1.2 Metryki oceny . . . 14
- 5.2 Przeprowadzenie analizy wyników oraz porównanie jakości odpowiedzi w obu trybach działania systemu . . . 14

Rozdział 1

Wstęp i cel pracy

Aktualnie duże model językowe (ang. LLM, Large Language Model) wykorzystywane są w różnych obszarach obszarach życia. Wysokie zainteresowanie sztuczną inteligencją sprzyja rozwojowi tej dziedziny. Skutkuje to pojawianiem się nowych możliwości wykorzystania technologii przetwarzania języka naturalnego oraz zrewolucjonizowaniem sposobu, w jakim systemy informatyczne przetwarzają i analizują dane nieustrukturyzowane.

W miarę jak rosną oczekiwania wobec systemów opartych na LLM, pojedynczy model może okazać się niewystarczający. Typowymi ograniczeniami są kontekst, koszt obliczeniowy pojedynczego zapytania czy podatność na błędy wynikające z braku weryfikacji własnych odpowiedzi. Naturalną odpowiedzią na te ograniczenia jest koncepcja architektur wielomodelowych, w których kilka instancji LLM współpracuje ze sobą.

Podejścia do organizacji współpracy modeli są różne. Często inspirowane są one istniejącymi systemami występującymi w przyrodzie lub życiu codziennym. Ciekawym podejściem wydaje się podejście umysłu rojowego (ang. hive mind).

1.1 Motywacja

Motywacją do realizacji projektu jest rosnące zainteresowanie sztuczną inteligencją, a w szczególności dużymi modelami językowymi oraz wykorzystaniem ich możliwości. Ciekawym zagadnieniem jest zbadanie wpływu współpracy wielu modeli na jakość odpowiedzi.

1.1.1 Dynamiczny rozwój dziedziny

W ciągu ostatnich dekad można było zauważyć dynamiczny postęp w obszarze sztucznej inteligencji (SI, ang. Artificial Intelligence, AI). Rozwój tej dziedziny doprowadził do znacznego zwiększenia liczby prowadzonych na całym świecie badań, co znajduje odzwierciedlenie między innymi w rosnącej liczbie cytowań publikacji poświęconych tej tematyce. Z przeprowadzonych analiz bibliometrycznych wynika, że w latach 2012–2022 liczba publikacji dotyczących sztucznej inteligencji oraz Big Data rosła wykładniczo, przy czym szczególnie wyraźne przyspieszenie tego trendu nastąpiło od 2019 roku. Rozwiązania oparte na SI znalazły zastosowanie w wielu różnych obszarach, między innymi w medycynie, edukacji, biznesie i zarządzaniu, usługach rządowych (inteligentne miasta), finansach (prognozowanie i analiza) [1], a także w turystyce oraz sektorze rozrywkowym.

Jedną z dziedzin rozwijających się w ramach sztucznej inteligencji jest przetwarzanie języka naturalnego (ang. Natural Language Processing, NLP). Jej głównym zadaniem jest umożliwienie komputerom pracy z językiem używanym przez człowieka poprzez jego odpowiednie

przekształcanie i interpretowanie. Pozwala to na automatyczną analizę, generowanie oraz przetwarzanie treści tekstowych. Istotnym elementem rozwoju NLP stały się również duże modele językowe (LLM), których możliwości w ciągu ostatnich lat znacząco wzrosły. Modele te ewoluowały z pasywnych systemów przetwarzania tekstu w aktywne podmioty (agenty AI) zdolne do autonomicznego planowania, rozumowania i wchodzenia w interakcje ze środowiskiem [2]. Ich popularność gwałtownie zwiększyła się w listopadzie 2022 roku, kiedy firma OpenAI udostępniła model ChatGPT-3.5. Narzędzie to w bardzo krótkim czasie zdobyło ogromną liczbę użytkowników - w nieco ponad dwa miesiące przekroczyło próg stu milionów użytkowników.

Rozwój dużych modeli językowych znajduje zastosowanie również w codziennym pozyskiwaniu informacji i rozwiązywaniu różnego rodzaju problemów. Generatywna sztuczna inteligencja (GenAI) może być wykorzystywana między innymi do wspomagania użytkowników w rozwiązywaniu zadań matematycznych, wyjaśniania zagadnień naukowych czy udzielania odpowiedzi na pytania związane ze zdrowiem i medycyną. Odpowiednio sformułowane zapytanie pozwala modelowi nie tylko przedstawić wynik, ale również wyjaśnić sposób jego uzyskania i dostosować poziom odpowiedzi do potrzeb użytkownika. Podobnie jak w przypadku innych zastosowań modeli językowych, istotnym elementem jest sposób formułowania promptów. Zbyt ogólne, nieprecyzyjne lub pozbawione odpowiedniego kontekstu polecenie może prowadzić do wygenerowania odpowiedzi niepełnej, błędnej lub niedopasowanej do oczekiwań użytkownika. Ponadto, aby ograniczyć zjawisko halucynacji LLM, stosuje się wielowarstwowe mechanizmy obronne, monitorowanie przepływu pracy (Workflow Monitoring) oraz generowanie ustrukturyzowanych odpowiedzi (JSON), co pozwala na lepszą kalibrację pewności modelu [1]. Jest to szczególnie istotne w przypadku dziedzin wymagających wysokiej dokładności, takich jak matematyka czy medycyna, gdzie odpowiedzi generowane przez modele powinny być traktowane jako forma wsparcia, a nie jako bezwzględnie wiarygodne źródło informacji.

1.1.2 Nowe możliwości

Dynamiczny rozwój sztucznej inteligencji oraz metod przetwarzania języka naturalnego przyczynił się do powstania nowych sposobów wykorzystania komputerów w codziennych zadaniach. Szczególne znaczenie mają tutaj generatywna sztuczna inteligencja oraz duże modele językowe, które ewoluowały z pasywnych systemów odpowiedzi w autonomiczne agenty AI. Umożliwiają one nie tylko uzyskiwanie informacji, ale również samodzielne korzystanie z zewnętrznych narzędzi, planowanie wieloetapowych zadań i interakcję ze środowiskiem w celu realizacji złożonych celów [3]. Rozwiązania te mogą być wykorzystywane między innymi do wspomagania nauki, rozwiązywania zadań matematycznych, wyjaśniania zagadnień z różnych dziedzin czy udzielania odpowiedzi na pytania wymagające analizy dostarczonych informacji.

Duże modele językowe

Duże modele językowe (ang. Large Language Models, LLM) w ostatnich latach stały się jednym z najważniejszych obszarów rozwoju sztucznej inteligencji. Jeszcze stosunkowo niedawno bezpośrednia komunikacja z komputerem za pomocą języka naturalnego wymagała specjalistycznych rozwiązań, natomiast obecnie użytkownik może formułować pytania i polecenia w sposób zbliżony do rozmowy z drugą osobą. Dzięki temu modele językowe mogą pełnić funkcję narzędzia wspomagającego, a obecnie coraz częściej tworzą złożone systemy wieloagentowe (ang. Large Language Model-based Multi-Agent Systems, MASs) [2], [3]. W takich strukturach wyspecjalizowane jednostki (np. agent-programista, agent-tester) współpracują ze sobą, co pozwala na naturalny podział pracy, większą odporność na błędy oraz rozwiązywanie

problemów o stopniu złożoności przekraczającym możliwości pojedynczego modelu. Przykładowe zastosowania obejmują generowanie i analizowanie tekstu, tłumaczenie treści, tworzenie podsumowań, odpowiadanie na pytania, a także pomoc w rozwiązywaniu problemów matematycznych i logicznych.

Istotną zaletą części dostępnych modeli językowych jest ich otwarty charakter (ang. open-source software), który umożliwia wykorzystanie ich we własnych aplikacjach. Platformy takie jak Hugging Face udostępniają szeroki wybór modeli, które można dostosować do określonych zastosowań i zintegrować z tworzonym oprogramowaniem. Pozwala to między innymi na budowanie systemów wykorzystujących modele językowe do analizy pytań użytkownika oraz generowania odpowiedzi bez konieczności tworzenia własnego modelu od podstaw.

Wykorzystanie modeli uruchamianych lokalnie daje również możliwość ograniczenia konieczności przesyłania danych do zewnętrznych usług. Przy odpowiednio wydajnej infrastrukturze model może zostać uruchomiony bezpośrednio na komputerze użytkownika, a komunikacja z nim może odbywać się za pośrednictwem lokalnego interfejsu API. Takie rozwiązanie może mieć szczególne znaczenie w przypadku aplikacji przetwarzających informacje, dla których istotne są prywatność i bezpieczeństwo. Lokalna obsługa modelu pozwala bowiem zachować dane w obrębie własnej infrastruktury, ograniczając ryzyko ich nieautoryzowanego udostępnienia.

1.2 Cel pracy

Celem pracy jest stworzenie systemu przetwarzania o nazwie MLLMS - Multi-Large Language Model System, który umożliwia efektywną współpracę dużych modeli językowych w celu rozwiązywania złożonych problemów. Projekt zakłada implementację takiego systemu. Głównym założeniem pracy jest zbadanie wpływu współpracy wielu modeli na jakość odpowiedzi oraz identyfikację najlepszych praktyk w zakresie projektowania i implementacji takich rozwiązań.

Aplikacja zostanie zaimplementowana w języku Python i będzie umożliwiała komunikację z lokalnie uruchomionymi dużymi modelami językowymi. Połączenie z modelami zostanie zrealizowane za pośrednictwem interfejsu API platformy Ollama, dzięki czemu przetwarzanie zapytań będzie odbywało się w całości na urządzeniu użytkownika, bez konieczności korzystania z zewnętrznych usług chmurowych. Modele biorące udział w działaniu systemu będą współdzieliły jedną wspólną przestrzeń, służącą do wymiany spostrzeżeń oraz wyników przeprowadzanych obliczeń. Dane wyjściowe poszczególnych modeli będą łączone w jeden ciąg znaków, który następnie stanie się kontekstem dostępnym dla wszystkich uczestników. Dzięki temu każdy z modeli będzie mógł odwoływać się do wniosków sformułowanych przez pozostałe, co pozwoli na wspólne rozwiązywanie problemów i wzajemne uzupełnianie się w trakcie pracy.

Interakcja z aplikacją będzie odbywała się z poziomu terminala za pomocą tekstowego interfejsu użytkownika (ang. Text User Interface, TUI). Interfejs zostanie zrealizowany bez użycia dedykowanych bibliotek do budowy interfejsów terminalowych, co pozwoli ograniczyć liczbę zależności oraz zachować niewielkie wymagania i prostotę uruchomienia charakterystyczną dla narzędzi działających w wierszu poleceń.

Projekt ma na celu zbadanie efektywności współpracy wielu dużych modeli językowych w rozwiązywaniu złożonych problemów oraz identyfikację najlepszych praktyk w zakresie projektowania i implementacji systemów opartych na współpracujących LLM. Aplikacja MLLMS inspirowana jest występującym w przyrodzie umysłem rojowym (ang. hive mind).

1.3 Struktura pracy

Praca liczy sześć głównych rozdziałów, ułożonych tak, aby krok po kroku poznawać zagadnienie oraz kolejne etapy realizacji projektu. W rozdziale pierwszym przedstawiono tematykę pracy i określono jej cel. Rozdział drugi ma charakter teoretyczny i wprowadza w omawianą dziedzinę, wyjaśniając pojęcie generatywnej sztucznej inteligencji oraz dużych modeli językowych. W rozdziale trzecim opisano technologie użyte w projekcie, do których należą Python, Ollama i modele językowe. Rozdział czwarty poświęcono projektowi systemu, w którym szczegółowo omówiono poszczególne komponenty. W rozdziale piątym zaprezentowano wyniki badań dotyczące skuteczności współpracy wielu modeli, wydajności modeli językowych oraz jakości generowanych przez nie odpowiedzi. Rozdział ostatni zawiera podsumowanie uzyskanych rezultatów oraz kierunki dalszego rozwoju pracy.

W tabeli 1.1 przedstawiono podział odpowiedzialności autorów za poszczególne rozdziały i podrozdziały.

Tabela 1.1: Podział odpowiedzialności autorów za poszczególne rozdziały

Autor	**Numery rozdziałów i podrozdziałów**
Olwier Komorowski	1, 3.1, 3.4
Jakub Nowak	2, 3.2, 3.3, 3.5, 4, 5, 6

Rozdział 2

Wprowadzenie do dziedziny

Niniejszy rozdział wprowadza w dziedzinę, w której osadzona jest praca. Najpierw przedstawia generatywną sztuczną inteligencję: jej definicję, zasadę działania, zastosowania oraz ograniczenia. Następnie omawia duże modele językowe, czyli klasę modeli generatywnych, na której opiera się system MLLMS. Wprowadzone pojęcia są podstawą dla kolejnych rozdziałów, które opisują technologie, projekt systemu i wyniki badań.

2.1 Generatywna sztuczna inteligencja

2.1.1 Definicja i miejsce generatywnej sztucznej inteligencji

Sztuczna inteligencja (SI) obejmuje metody, które pozwalają komputerom wykonywać zadania wymagające zwykle ludzkiej inteligencji. Jej istotną częścią jest uczenie maszynowe (ang. Machine Learning, ML), w którym zależności nie są programowane wprost, lecz wyznaczane przez algorytm na podstawie danych. Szczególnym przypadkiem uczenia maszynowego są sztuczne sieci neuronowe (ang. Artificial Neural Networks), czyli modele zbudowane z warstw prostych jednostek obliczeniowych, których parametry dobierane są w procesie uczenia. Sieci składające się z wielu warstw określa się mianem uczenia głębokiego (ang. Deep Learning, DL). Umożliwiło ono modelowanie bardzo złożonych zależności w danych, takich jak tekst, obraz czy dźwięk.

Na uczeniu głębokim opiera się współczesna generatywna sztuczna inteligencja (ang. Generative Artificial Intelligence, GenAI). Według [4] są to techniki obliczeniowe zdolne do generowania pozornie nowych, sensownych treści, takich jak tekst, obrazy czy dźwięk, na podstawie danych treningowych. Modele generatywne nie ograniczają się więc do klasyfikowania danych wejściowych lub przewidywania pojedynczej wartości, lecz tworzą nowe dane, przypominające te, na których były uczone. Wśród nich szczególne miejsce zajmują duże modele językowe, opisane w sekcji 2.2.

2.1.2 Zasada działania i zastosowania

Modele generatywne uczą się rozkładu prawdopodobieństwa danych treningowych $p(x)$, a w wersji warunkowej rozkładu $p(x \mid c)$, w którym c oznacza kontekst, na przykład polecenie użytkownika (ang. prompt). Generowanie polega na losowaniu (ang. sampling) kolejnych elementów z tego rozkładu. W modelach językowych elementami są tokeny, czyli fragmenty tekstu, a odpowiedź powstaje token po tokenie: każdy kolejny wybierany jest na podstawie zapytania i dotychczas wygenerowanego tekstu. Stopień losowości reguluje parametr zwany temperaturą

(ang. temperature). Dla tego samego zapytania model może więc zwrócić różne odpowiedzi, a ich powtarzalność zapewnia dopiero ustalone ziarno generatora liczb losowych (ang. seed).

Modele dyskryminatywne uczą się natomiast przypisywać dane wejściowe do kategorii, czyli modelują $p(y \mid x)$, gdzie y jest etykietą [5]. Typowym przykładem jest filtr spamu rozstrzygający, czy wiadomość jest niechcianą pocztą. Pytanie wielokrotnego wyboru wygląda jak zadanie klasyfikacji, jednak model generatywny nie wybiera klasy, lecz generuje tekst, w którym znajduje się litera odpowiedzi. Jego odpowiedź jest więc wynikiem losowania, a nie deterministycznej decyzji klasyfikatora.

Treści tworzone przez modele generatywne mogą mieć różną postać. Według [4] są to między innymi tekst, obrazy i wideo, mowa i muzyka oraz kod źródłowy. Niniejsza praca dotyczy wyłącznie tekstu.

Generatywna sztuczna inteligencja jest wykorzystywana w edukacji, nauce, medycynie, biznesie i codziennej pracy z informacją, między innymi do odpowiadania na pytania, streszczania i tłumaczenia tekstów, wyjaśniania zagadnień oraz pomocy w programowaniu. Szczególnie istotne dla tej pracy są zadania wymagające rozumowania i wiedzy specjalistycznej, takie jak rozwiązywanie zadań matematycznych i odpowiadanie na pytania medyczne, na których oparto badania opisane w dalszej części pracy. W tych dziedzinach wiarygodność odpowiedzi ma duże znaczenie, co prowadzi do ograniczeń omówionych w następnej podsekcji.

2.1.3 Ograniczenia generatywnej sztucznej inteligencji

Mimo szerokich możliwości generatywna sztuczna inteligencja ma ograniczenia, które wynikają bezpośrednio ze sposobu jej działania.

Najbardziej znanym z nich są halucynacje (ang. hallucinations), czyli generowanie treści brzmiących wiarygodnie, lecz nieprawdziwych. Wyróżnia się wśród nich halucynacje dotyczące faktów, gdy treść jest sprzeczna z weryfikowalną wiedzą o świecie, oraz halucynacje dotyczące wierności, gdy odpowiedź odbiega od polecenia lub kontekstu [6]. Ponieważ model losuje tokeny zgodnie z wyuczonym rozkładem i nie sprawdza prawdziwości treści, odpowiedź błędna wygląda tak samo jak poprawna, a przekonujący ton utrudnia wykrycie błędu. Ma to duże znaczenie w dziedzinach takich jak medycyna, w których błędna informacja może prowadzić do szkody.

Z tym wiąże się brak samoweryfikacji. Model nie posiada wbudowanego mechanizmu sprawdzania własnej odpowiedzi, a poproszenie go o ponowną ocenę własnego rozumowania nie musi prowadzić do poprawy. Pojedynczy model jest więc jednocześnie autorem i jedynym recenzentem odpowiedzi.

Kolejnym ograniczeniem jest wrażliwość na sformułowanie polecenia. Słaby prompt, czyli ogólnikowy lub pozbawiony kontekstu, zwykle daje odpowiedź niepełną lub błędną, natomiast dobrze zbudowany, precyzujący zadanie i oczekiwany format, poprawia wynik. Nawet drobne zmiany formy polecenia, niezmieniające jego sensu, mogą zmienić odpowiedź.

Ograniczenia te, w szczególności brak samoweryfikacji, stanowią punkt wyjścia dla koncepcji współpracy wielu modeli, opisanej w kolejnych rozdziałach.

2.2 Duże modele językowe

Rozdział 3

Technologie i narzędzia

Istotny był wybór odpowiednich technologii i narzędzi. Ważne było aby system był wydajny i jego poszczególne komponenty łatwe do wdrożenia dzięki odpowiednim udokumentowaniu. Celem tego rozdziału jest przedstawienie użytych w projekcie technologii oraz uzasadnienie ich wyboru.

3.1 Wybrane języki programowania

Do realizacji projektu zdecydowano się na język programowania Python, który dzięki swojej prostocie, szybkości implementacji oraz sporej liczbie bibliotek, był najlepszym wyborem z pośród innych języków. Dodatkowym atutem jest jego popularność pośród istniejących już rozwiązań uczenia maszynowego. Stanowiło to dodatkową inspirację podczas projektowania systemu.

3.2 Biblioteki

W projekcie zostały wykorzystane różne biblioteki, które pozwoliły przyspieszyć czas implementacji, zmniejszając potrzebę pisania kodu od podstawy oraz utrzymania wysokiej jakości rozwiązań problemów programistycznych.

- json - random - re - requests - subprocess

3.3 Wybrane modele językowe

Dobór modeli podyktowany był założeniami projektu. System opiera się na współpracy dwóch modeli. Jej wpływ na jakość odpowiedzi podlega badaniu, dlatego potrzebne były modele pochodzące od różnych twórców, a nie dwie instancje tego samego modelu. Ponadto modele musiały z przyzwoitą dokładnością rozwiązywać zadania z obu dziedzin, na których oparto badania, czyli matematyki i medycyny. Kolejnym wymogiem było, aby były to modele o otwartych wagach (ang. open-weight), dostępne w bibliotece Ollamy i możliwe do uruchomienia lokalnie. Ostatnim kryterium było ograniczenie sprzętowe – oba modele musiały mieścić się w pamięci operacyjnej komputera, na którym prowadzono badania (szczegóły opisano w sekcji 3.5). W ramach tego ograniczenia wybierano modele o możliwie największej liczbie parametrów, ponieważ jest ona jednym z czynników wpływających na jakość generowanych odpowiedzi.

Tabela 3.1 zestawia modele rozważane na etapie wyboru. Wytłuszczono modele użyte w projekcie.

Tabela 3.1: Modele rozważane w projekcie

Model	Twórca	Parametry	Decyzja i powód
gemma4:12b	**Google**	**12 mld**	**Wybrany.** Najnowsza generacja rodziny Gemma, dobra jakość na zadaniach obu dziedzin, mieści się w limicie pamięci.
qwen3.5:9b	**Alibaba**	**9 mld**	**Wybrany.** Najnowsza generacja rodziny Qwen, dobra jakość na zadaniach obu dziedzin, inna rodzina niż Gemma.
llama3.1:8b	Meta	8 mld	Odrzucony. Wyraźnie słabsza jakość odpowiedzi.
qwen2.5:7b	Alibaba	7 mld	Odrzucony. Jakość przeciętna oraz starsza generacja względem Qwen3.5.
gemma2:9b	Google	9 mld	Odrzucony. Jakość przeciętna oraz starsza generacja względem Gemma 4.
gemma2:2b	Google	2 mld	Odrzucony. Częstsze błędy i trudności z zachowaniem wymaganego formatu odpowiedzi.
phi4-mini	Microsoft	3,8 mld	Odrzucony. Częstsze błędy i trudności z zachowaniem wymaganego formatu odpowiedzi.
deepseek-r1:14b	DeepSeek-AI	14 mld	Odrzucony. Bardzo dobre wyniki w matematyce, zbliżone do wybranych modeli, ale wyższe zużycie pamięci.

Wybrane zostały modele gemma4:12b (Google) oraz qwen3.5:9b (Alibaba). Pochodzą od różnych twórców i należą do różnych rodzin, co pozwala na zróżnicowane odpowiedzi i rozumowanie. Mają przy tym zbliżony rozmiar, dzięki czemu ewentualne różnice w wynikach nie wynikają wyłącznie z liczby parametrów. Oba reprezentowały najnowszą generację swoich rodzin w momencie wyboru.

Wybór opierał się na wstępnych próbach przeprowadzonych na działającej aplikacji, a nie na systematycznym porównaniu. Jakość wybranych modeli zbadano w rozdziale 5.

3.4 Narzędzia

Git

Git to narzędzie służące do zarządzania wersjami projektu, umożliwiające monitorowanie wprowadzanych zmian w kodzie oraz równoczesną pracę wielu osób nad tym samym projektem.

W projekcie pełnił funkcję podstawowego systemu kontroli wersji. Każdy członek zespołu pracował na oddzielnej gałęzi (ang. branch), a po zakończeniu prac wprowadzone zmiany były łączone z główną gałęzią projektu (ang. main branch). Takie podejście ułatwiało współpracę, ograniczało ryzyko występowania konfliktów oraz pozwalało na zachowanie uporządkowanej, liniowej historii zmian.

GitHub

GitHub to platforma wykorzystująca system Git, która pozwala na przechowywanie oraz zarządzanie repozytoriami znajdującymi się na zdalnych serwerach. Oprócz podstawowych funkcji oferuje również możliwość wzajemnego przeglądania i weryfikowania kodu przez członków zespołu, a także automatycznego uruchamiania procesów sprawdzających poprawność projektu, takich jak wykonywanie testów czy weryfikacja kompilacji kodu w ramach CI (ang. Continuous Integration). Funkcja przeglądu kodu pozwalała zespołowi na bieżąco kontrolować jakość wprowadzanych zmian, wykrywać potencjalne błędy oraz utrzymywać kod w odpowiednim stanie.

Ollama API

Ollama to oprogramowanie umożliwiające lokalne uruchamianie oraz zarządzanie wybranymi modelami językowymi typu LLM (ang. Large Language Model).

W projekcie zostało wykorzystane do obsługi modeli działających bezpośrednio na lokalnej infrastrukturze, co pozwalało na korzystanie z ich możliwości bez konieczności przesyłania danych do zewnętrznych usług. Komunikacja pomiędzy aplikacją a uruchomionymi modelami odbywała się za pośrednictwem interfejsu API, umożliwiając przesyłanie zapytań oraz odbieranie wygenerowanych odpowiedzi.

3.5 Wymagania sprzętowe

Rozdział 4

Projekt systemu

4.1 Implementacja systemu przetwarzania z wykorzystaniem pojedynczego modelu LLM

4.2 Rozszerzenie systemu do wariantu z dwoma współpracującymi modelami LLM

Rozdział 5

Badania i analiza

5.1 Wybór benchmarków oraz metryk oceny jakości odpowiedzi

5.1.1 Benchmarki

5.1.2 Metryki oceny

5.2 Przeprowadzenie analizy wyników oraz porównanie jakości odpowiedzi w obu trybach działania systemu

Spis tabel

1.1 Podział odpowiedzialności autorów za poszczególne rozdziały 7

3.1 Modele rozważane w projekcie 11

Spis rysunków

Bibliografia

[1] Y. Cheng, Y. Xu, C. Yu, Y. Zhao, "HAWK: A Hierarchical Workflow Framework for Multi-Agent Collaboration," *arXiv preprint arXiv:2507.04067*, 2025. URL: https://arxiv.org/abs/2507.04067

[2] Y. Yang, Q. Peng, J. Wang, Y. Wen, W. Zhang, "LLM-based Multi-Agent Systems: Techniques and Business Perspectives," *arXiv preprint arXiv:2411.14033*, 2024. URL: https://arxiv.org/abs/2411.14033

[3] K.-T. Tran, D. Dao, M.-D. Nguyen, Q.-V. Pham, B. O'Sullivan, H. D. Nguyen, "Multi-Agent Collaboration Mechanisms: A Survey of LLMs," *arXiv preprint arXiv:2501.06322*, 2024. URL: https://arxiv.org/abs/2501.06322

[4] S. Feuerriegel, J. Hartmann, C. Janiesch, P. Zschech, "Generative AI," *Business & Information Systems Engineering*, t. 66, nr 1, s. 111–126, 2024. URL: https://arxiv.org/abs/2309.07930

[5] A. Y. Ng, M. I. Jordan, "On Discriminative vs. Generative Classifiers: A Comparison of Logistic Regression and Naive Bayes," w: *Advances in Neural Information Processing Systems 14 (NIPS 2001)*, 2001. URL: https://proceedings.neurips.cc/paper/2001/hash/7b7a53e239400a13bd6be6c91c4f6c4e-Abstract.html

[6] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, T. Liu, "A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions," *ACM Transactions on Information Systems*, t. 43, nr 2, s. 1–55, 2025. URL: https://arxiv.org/abs/2311.05232